Astro lab

FYSB23

Author
Fredrik Bergelv
fredrik.bergelv@live.se

Lund University, Department of Physics

under the direction of
Ross Church



LUND UNIVERSITY

September 15, 2024

Contents

1 Introduction **2**

2 Theory **2**

3 Exercises **3**

3.1 Working out the conversion constant 3

3.2 Plotting of the spectrum 3

3.3 Calculating the strength of the line 3

3.4 Tabulating all the strengths 4

3.5 Plotting of excitation potential 4

3.6 Calculating the temperature 5

3.7 Reflection on the difficulties 5

A The Python code **7**

1 Introduction

All stars emit photons which interact with the atoms which it consists of. This will lead to dark lines in the spectrum from the star corresponding to the absorbtion wavelengths of the atoms. By calculating the strength of each line one can calculate the number of electrons situated in different levels of the atom, and thus calculate the temperature of the star. The goal of this lab was to retrieve the temperature of star from given data, and thus conclude which star the data corresponds to.

2 Theory

Stars can to a certain degree be viewed as a black body which radiates light depending on the temperature. However, the star consists of atoms which will absorb light of certain wavelengths. Consequently, one could see certain black lines in the spectrum of the star since some wavelengths are absorbed by the atoms.

To calculate the strength of a line in a spectrum one can use the relative flux which is defined as the integral of the deviation from the continuum flux. One can thus define the strength of each line by

$$S = \int_{\lambda_-}^{\lambda_+} [1 - F(\lambda)]\, d\lambda, \tag{1}$$

where $F(\lambda)$ is the relative flux and λ_- and λ_+ is the width of the line [1].

This line strength is directly related to to the transitions of the atom. By using quantum statistics and the formula of the number of electrons in a certain levels n one can evaluate the temperature. The number of electrons in a certain levels n is described as

$$N_n \propto g_n e^{-\frac{E_n}{k_B T}}, \tag{2}$$

where g_n is the statistical weight, E_n is the energy, k_B is the Boltzmann constant and T is the temperature [1].

With the equations above one can retrieve an expression for the temperature T depending on the wavelength λ and the the strength S as

$$\log_{10}\left(\frac{S}{\lambda}\right) = \log_{10}\left(\frac{cN}{k_B Z}\right) + \log_{10}(gf\lambda) - \chi_n \frac{\log_{10}(e)}{k_B T}. \tag{3}$$

Above c is the speed of light, N is the number of particles, Z is the partian function and e is Euler's constant [1].

3 Exercises

3.1 Working out the conversion constant

The constant $\log_{10}(e)/k_B$ was calculated to $5039.7781\,\mathrm{K\,eV^{-1}}$.

3.2 Plotting of the spectrum

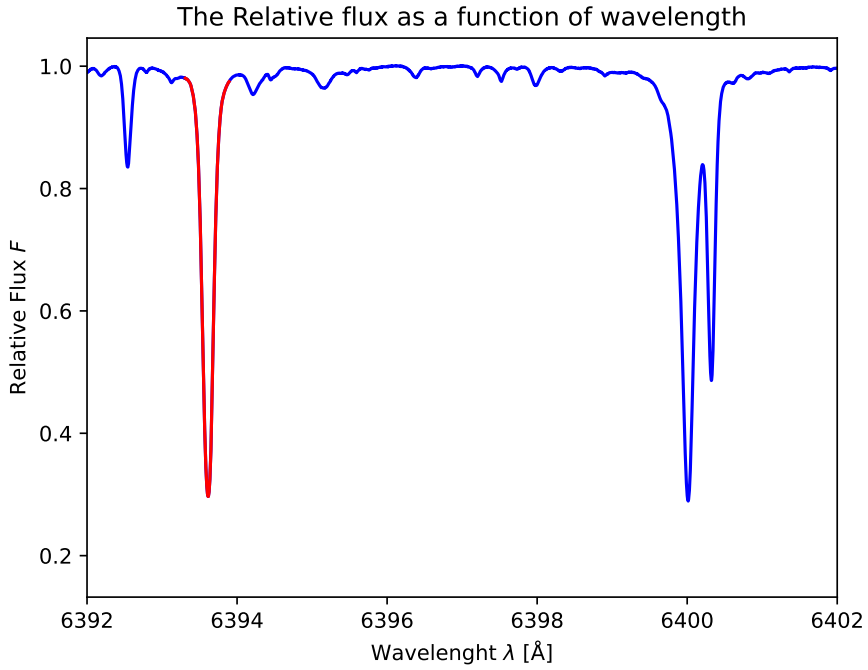


Figure 1: This figure shows the relative flux of the data from `spectrum.dat` at different wavelengths. The line which was uses is marked in red.

The reason why the red line was used is because this line is not a double line as the right line. A double line becomes much more difficult to calculate since it includes two different lines, with different strengths. Furthermore can one see from `LineList.dat` that the left line is included, while the right is not. This might indicate that the right line

3.3 Calculating the strength of the line

The limits in Equation 1 was chosen by viewing Figure 1. Thus, the wavelengths that was used were a lower limit of 6393.340 Å and am upper limit of 6393.928 Å. This can be seen as the region marked in red in Figure 1. One can see that this region shows this line, while excluding the flux from the small lines close to this one. This gave the calculated strength using Equation 1 to 0.1377 Å.

3.4 Tabulating all the strengths

The tabulated strengths form `LineList.dat` using Equation 1 can be viewed below.

Line λ [Å]	Strength S [Å]
6393.6004	0.1377
6416.9331	0.0415
6430.8450	0.1185
6481.8698	0.0678
6494.9804	0.1690
6496.4656	0.0659
6498.9383	0.0474
6518.3657	0.0595
6533.9281	0.0557
6546.2381	0.1125
6574.2266	0.0347

Table 1: The tabulated result from `LineList.dat` with there corresponding strengths.

3.5 Plotting of excitation potential

The reason why it is god to plot

$$\log_{10}\left(\frac{S}{\lambda}\right) - \log_{10}\left(gf\lambda\right) \tag{4}$$

against the excitation potential χ is because this plot will give information about Equation 3. By observing Equation 3 one can tell that this equation resembles a linear regression of χ against the expression above, where $\log_{10}\left(cN/k_BZ\right)$ is only a constant. Thus, this plot would give insight into the value of $-\log_{10}(e)/k_BT$, which would give a negative slope of the regression (Since the temperature is positive). Consequently, one could calculate the temperature from this slope.

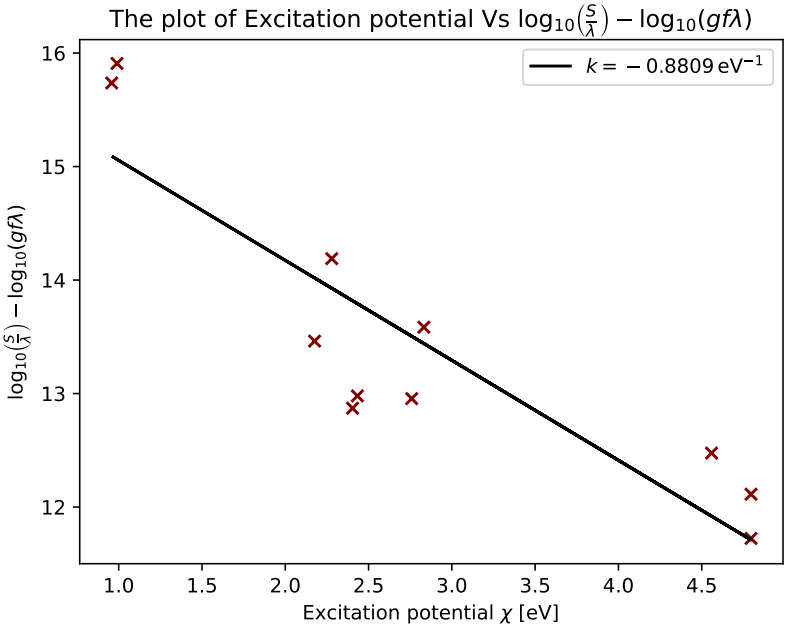


Figure 2: This figure shows the linear regression on the form $y = kx + m$, where k is displayed.

3.6 Calculating the temperature

The slope gave a value of $-0.8809\,\mathrm{eV}^{-1}$. Since we know the that the slope is described as $-\log_{10}(e)/k_BT$ one can calculate the temperature. The calculated surface temperature is thus 5721 K. The photosphere temperature of the Sun is 5772 K [2]. This means that the deviation from our calculated temperature form the temperature of the Suns is 0.8810%. It is thus reasonable to conclude the the observed star is the Sun.

3.7 Reflection on the difficulties

There may be errors in the given data that affect the precision of the result. For example is one error the fact that the spectra includes lines from other elements, making it more complicated to identify the lines. Other errors, such errors in the measurements from the data-files may also affect the given data. For example is the temperature of the surface of the sun not discrete. The formulas which are used however assume that the temperatures are constant. The Sun is fluid where the surface is ever changing. Furthermore is the suns surface not a direct surface, but more like a mist. This is an error which affects the data and thus the final temperature.

One could see that there was some trouble determining the integration limits in Figure 1. Fitting the integration limits could be done more effectively and thus obtaining a better result. Fitting these were consequently harder since the lines blended together. It was therefor harder to determine the limits, since some of the lines where closes placed to other lines. Since the lines resemble Gaussian functions could one solution be to fit Gaussian expressions on the lines. From this would it be easier to determining the different lines and the integrating limits. If this method was implemented we would reach integrals which would be larger since the integration limits does not have to be cut of due to line blending. Thus would the strengths S be larger and a higher temperature would be achieved. Since the calculated temperature is a lower than the real temperature would this give a more precise value. Thus would this improve the result.

Overall can one conclude that a value of 5721 K is a good since the deviation from the real temperature is 0.8810%.

References

[1] Ross Church and Alvaro Segovia Otero. *Taking the Temperature of a Star*. Lund University, 2024.

[2] D. R. Williams. Sun fact sheet. `https://nssdc.gsfc.nasa.gov/planetary/factsheet/sunfact.html`, July 1 2013. NASA Goddard Space Flight Center.

A The Python code

```
"""
Created on Thu Feb 22 15:43:32 2024
@author: Fredrik Bergelv
"""

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import scipy as sci

M = 0.5*2*10**30
h = 6.62607015*10**(-34)
me = 9.1093837*10**(-31)
mp = 1.67262192*10**(-27)
G = 6.6743*10**(-11)
e, kB, c = 1.602176634*10**(-19), 8.6173303*10**(-5), 299792458

conversion = (np.log10(np.e)/sci.constants.Boltzmann)*e

def read(file_path):
    with open(file_path, 'r') as file:
        next(file)
        data = [line.strip().split() for line in file]

        if len(data[0]) == 3:  # Check if there are three columns in the first line
            lambd = [float(entry[0]) for entry in data]  # This is in Å
            loggf = [float(entry[1]) for entry in data]
            chi = [float(entry[2]) for entry in data]  # This is in eV
            return lambd, loggf, chi
        else:
            lambd = [float(entry[0]) for entry in data]  # This is in Å
            F_per_Fc = [float(entry[1]) for entry in data]
            return lambd, F_per_Fc

lamb1, F = read("C:/Users/fredr/Downloads/spectrum.dat")
lamb2, loggf, chi = read("C:/Users/fredr/Downloads/LineList.dat")
F = np.array(F)
lamb1 = (np.array(lamb1))

plt.figure(1)
plt.plot(lamb1,F, c='b')
plt.xlim(6392,6402)
plt.xlabel(r'Wavelenght $\lambda$ [Å]')
plt.ylabel(r'Relative Flux $F$')
plt.title('The Relative flux as a function of wavelength')


lamb_neg, lamb_pos = 6393.340, 6393.928 # This was an estimation from the graph

neg = abs(np.array(lamb1)-lamb_neg)
pos = abs(np.array(lamb1)-lamb_pos)
for i in range(len(neg)):
    if neg[i] < 0.001 :
        upper = i
    if pos[i] < 0.0007 :
```

```
        lower = i

index = np.zeros(len(lamb2))
for k in range(len(lamb2)):
    for i in range(len(lamb1)):
        if abs(lamb1[i]-lamb2[k])<0.001:
            index[k]=int(i)

def strenght(lim, F=F, lamb1=lamb1):
    upper = lim[1]
    lower = lim[0]
    return abs(sci.integrate.simpson(1-F[lower:upper],lamb1[lower:upper]))


def limit (k, index=index, F=F, lamb1=lamb1):
    i = int(index[k])
    deviation = round(((upper-lower)/2)*F[int(index[0])]/F[int(index[k])])
    limlow = int(i - deviation)
    limup = int(i + deviation)
    return np.array([limlow, limup])

S = np.zeros(len(lamb2))
for k in range(len(lamb2)):
    plt.plot(lamb1[limit(k)[0]:limit(k)[1]],F[limit(k)[0]:limit(k)[1]], c='r')
    S[k] = strenght(limit(k))

#plt.savefig(r'C:\Users\fredr\OneDrive\Dokument\Python Scripts\Pictures\Wavelength_of_sun.pdf')

def expression(i, lamb2=lamb2, loggf=loggf, S=S):
    S = S[i]
    lamb = lamb2[i]*10**(-10)
    return np.log10(S/lamb)-loggf[i]-np.log10(lamb)

array = np.zeros(len(chi))
for i in range(len(chi)):
    array[i] = expression(i)

chi = np.array(chi)
plt.figure(2)
plt.scatter(chi, array, color='maroon', marker='x')
a = sci.stats.linregress(chi, array)


plt.plot(chi, a[0]*chi+a[1],
         label = f'$k = {round(a[0], 4)} \\, \\mathrm{{eV^{{-1}}}}$',
         color='black', linestyle=':')
plt.legend()
plt.xlabel(r'Excitation potential $\chi$ [eV]')
plt.ylabel(r'$\log_{10}\left(\frac{S}{\lambda}\right)-\log_{10}\left(gf\lambda\right)$')
plt.title(r'The plot of Excitation potential Vs $\log_{10}\left(\frac{S}{\lambda}\right)-\log_{10}\le
#plt.savefig(r'C:\Users\fredr\OneDrive\Dokument\Python Scripts\Pictures\Temp_plot.pdf')

T = -conversion/a[0]
diff = 100*abs(5772-T)/5772
print('The tempature at the surface is {round(T,4)} K' )
print(f'which is a deviation of {round(diff,4)} % from the real tempertaure')
```